© Guven

This notebook is provided in HTML format with an arbitrary saved widget state. Instructions: After downloading the file, copy the Code cells to a new Jupyter notebook and execute.

Perceptron Visualization

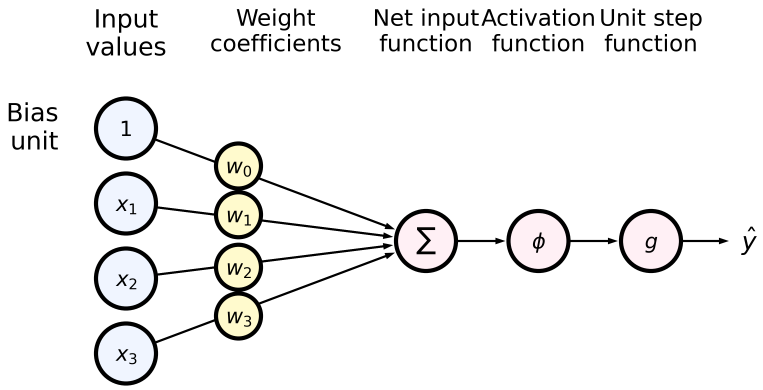


The perceptron is a binary classifier, an M dimensional hyperplane w, which separates the data points X into two categories $\{0, 1\}$, given by the ground truth y.

$$\mathbf{f}(x) = \hat{y} = \begin{cases} 1, & \text{if } w \cdot x + b > 0 \\ 0, & \text{otherwise} \end{cases}$$

Finding out this hyperplane or w, the perceptron training algorithm, such as Adaline, updates w by iteratively computing the classification error: $w \leftarrow w + \Delta w$, where $\Delta w = -\eta \nabla J(w)$ and $\nabla J(w)$ is the cost function and η is the learning rate.

The gradient of J, or $\nabla J(w)$ is computed by partial derivatives:



$\frac{\partial}{\partial w_j} J(w) = -\sum_i \left(y^{(i)} - a^{(i)}\right) x_j^{(i)}$, for $j = 1, \ldots, M$ and $i = 1, \ldots, N$

Activation function $\phi(z)$ where the net input z is the linear combination of the weights that are connecting the input to the output: $z = \sum_j w_j x_j = w^\top x$
The threshold function $g(z)$ is just a 0-threshold or binary step:
$\hat{y} = \begin{cases} +1, & \text{if } g(z) \geq 0 \\ -1, & \text{otherwise} \end{cases}$

In the visualization below, M is 2 as in the x_1 and x_2 axes, shown as the x and y dimensions in the plot.

In this example, to show properly, the perceptron equation $w \cdot x + b$, or $w^\top x + b$, defines the hyperplane in 2 dimensions. The hyperplane is all the points $x = (x_1, x_2)$ that make the hyperplane equation zero, i.e. $w^\top x + b = 0$

In the visualization below, the parameter theta (in radians) controls the angle of the hyperplane so that the w_1 and w_2 is computed from the given theta, b is the hyperplane offset and all the points x that make $w^\top x + b > 0$ are shown with the marker `+` and all the points x that make $w^\top x + b \leq 0$ are shown with the marker `o`.

Also shown is the **normal** of the hyperplane as red arrows.

In [1]:
```
%matplotlib inline
import matplotlib.pyplot as plt
plt.rcParams["figure.dpi"] = 72
from numpy import array, sin, cos, dot, pi, arange, linspace, meshgrid
from ipywidgets.widgets import interactive, interact, interactive_output, HBox, V

# Hyperplane xw+b=0
def hyper(x1, w, b):
    # x1, x2 are scalar/vector, w is vector, b is scalar
    # Find and return x2 from x1 that is on the hyperplane, i.e. solve the equati
    return -(x1*w[0]+b)/w[1] if w[1] != 0 else 0

# Classifier
def perceptron(x, w, b):
    # x, w are vectors, b is scalar
    # Return the classification, i.e. f(x)
    return 1 if dot(x,w)+b > 0 else 0
```

In [2]:
```
# h controls the display granularity
h = 1.7

# Display x1 and x2 limits
x1_min, x1_max = -10, 10
x2_min, x2_max = -10, 10

# Widgets in decorator
@interact(theta=FloatSlider(value=pi/8, min=-pi/1.7, max=pi/1.7), b=(-7.,7.))
def plot_w(theta, b):
    plt.figure(figsize=(6, 6), dpi=72)

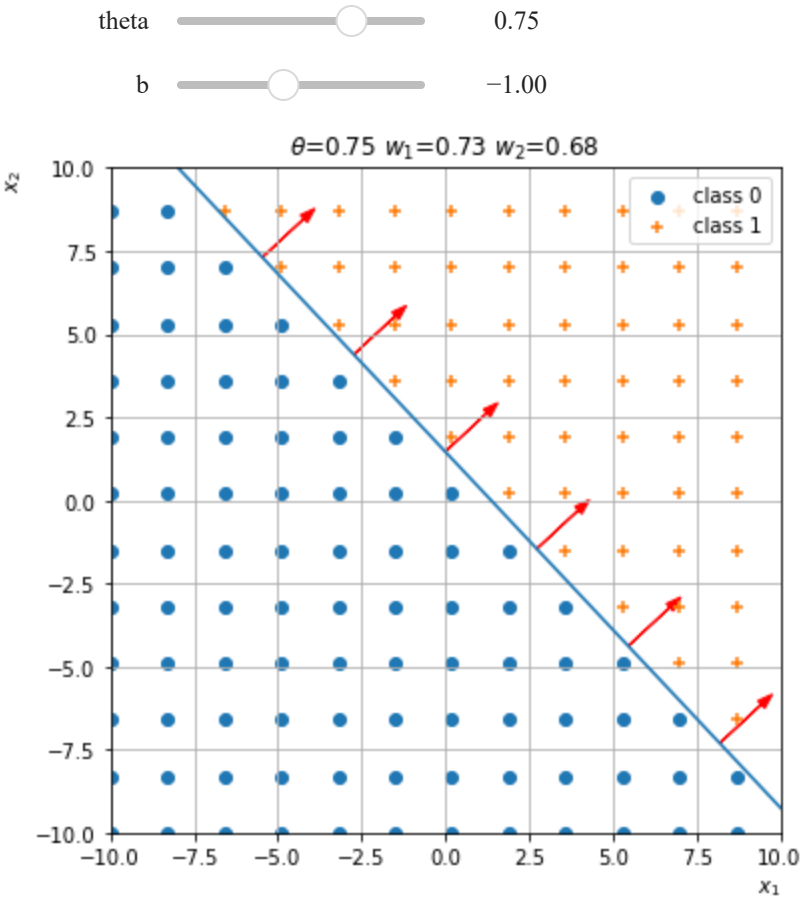
    # theta controls the angle of the hyperplane
    w = array([cos(theta), sin(theta)])

    # Plot normal arrows
    x1 = linspace(2*sin(theta)*x1_min, 2*sin(theta)*x1_max, 11)
    x2 = hyper(x1, w, b)
    x1_end = x1 + h*w[0]
    x2_end = x2 + h*w[1]
    for x1_o,x2_o,x1_e,x2_e in zip(x1,x2,x1_end,x2_end):
        plt.arrow(x1_o, x2_o, x1_e-x1_o, x2_e-x2_o, head_width=.3, color='r')
```

```
    # Plot the hyperplane
    x1 = arange(x1_min-h,x1_max+h,h)
    x2 = hyper(x1, w, b)
    plt.plot(x1, x2)
    plt.xlim(x1_min, x1_max)
    plt.ylim(x2_min, x2_max)
    plt.grid()

    # Show x1 and x2 points according to their classification by the perceptron,
    x1 = arange(x1_min-h,x1_max+h,h)
    x2 = arange(x2_min-h,x2_max+h,h)
    x1v, x2v = meshgrid(x1, x2, sparse=False, indexing='xy')
    p1 = [(x,y) for (x,y) in zip(x1v.flatten(), x2v.flatten()) if perceptron(arra
    plt.scatter(*zip(*p1), marker='o', label='class 0')
    p2 = [(x,y) for (x,y) in zip(x1v.flatten(), x2v.flatten()) if perceptron(arra
    plt.scatter(*zip(*p2), marker='+', label='class 1')
    plt.xlabel(r'$x_1$', horizontalalignment='right', x=1.0)
    plt.ylabel(r'$x_2$', horizontalalignment='right', y=1.0)
    plt.legend()

    # Hyperplane w
    plt.title(f'$\\theta$={theta:.2f} $w_1$={cos(theta):.2f} $w_2$={sin(theta):.2
    plt.show()
```



Study the above widget with different θ and b values. In supervised learning, this perceptron classifier is *trained* with the labels in y, to determine w_1, w_2, and

b by minimizing the classification error. An example training program is provided in the Textbook (Raschka, 2019) Chapter 2, section Training a perceptron model on the Iris dataset.

In [3]:
```
%%html
<style>
    table {margin-left: 0 !important;}
    p {font-family: verdana;}
    li {font-family: verdana;}
    div {font-size: 10pt;}
</style>
<!-- Display markdown tables left oriented in this notebook. -->
```
